

Lógica digital e organização de computadores

Pipeline e Paralelismo



Tecnologia em Análise e
Desenvolvimento de Sistemas

Campus de Alegre



O que é paralelismo?

- Paralelismo: É a execução de múltiplas tarefas em paralelo para aumentar a velocidade de processamento.
- Tipos:
 - Paralelismo de Instrução: Execução de múltiplas instruções de um único programa (Pipeline, Super-escalar).
 - Paralelismo de Processo: Execução de múltiplos programas (Multiprocessamento, Multicore).

A Necessidade de Paralelismo

- Desafio: O tempo de acesso à memória (latência) e o limite da frequência do clock (velocidade da UCP) limitam o ganho de performance.
- Solução: Em vez de fazer uma instrução mais rápida, fazemos múltiplas instruções simultaneamente.

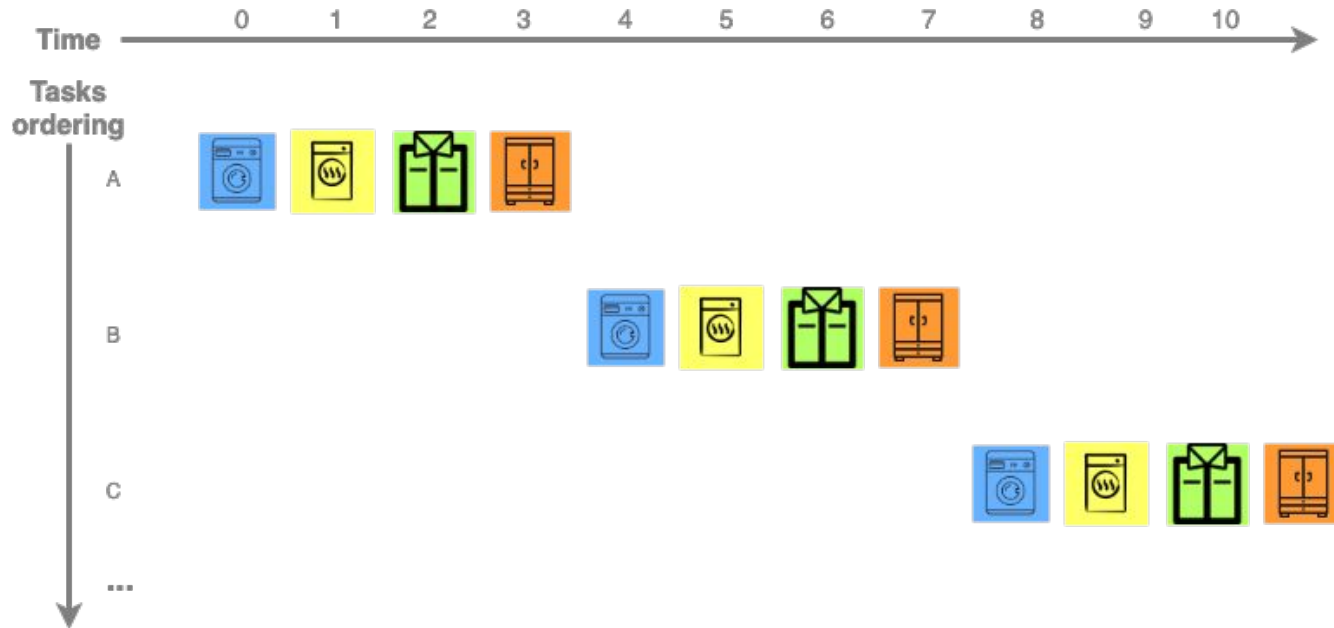
Execução Sequencial vs. Pipelined

- Execução Sequencial (Não-Pipelined): Uma instrução deve ser completamente concluída (Busca, Decodificação, Execução) antes que a próxima instrução possa começar.
- Tempo Total = $N \text{ instruções} \times \text{Tempo do Ciclo de Instrução}$.
- Execução Pipelined: O ciclo de instrução é dividido em estágios, permitindo que múltiplas instruções estejam em diferentes estágios de processamento ao mesmo tempo.

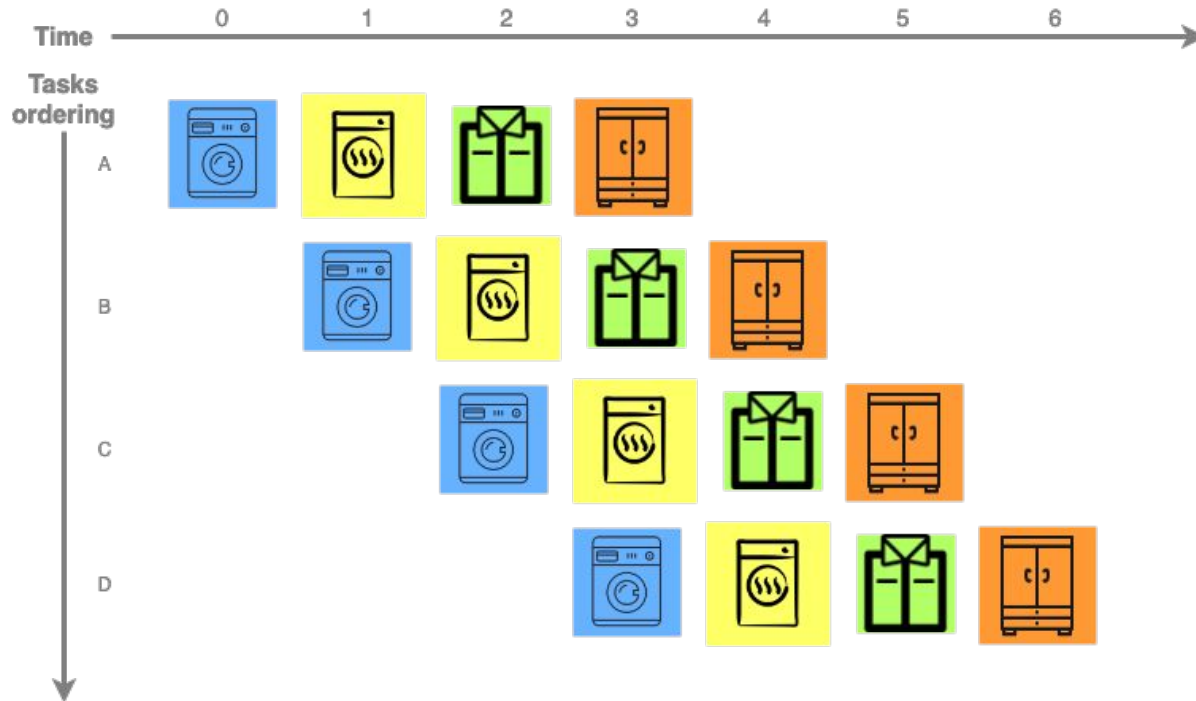
Conceito de Pipeline (Linha de Montagem)

- Analogia: Uma lavanderia. Cada equipamento (estágio) realiza uma tarefa específica em um item (instrução) e o passa para o próximo.
- Vantagem: O tempo para finalizar uma única instrução (Latência) é o mesmo (ou até ligeiramente maior).
- Ganho: A taxa de conclusão (Throughput ou Vazão) aumenta dramaticamente, pois uma nova instrução pode ser concluída a cada ciclo de clock.

Execução Sequencial vs. Pipelined



Execução Sequencial vs. Pipelined



Performance do Pipeline

- No geral, pode-se dizer que o número de estágios de um pipeline representa o ganho em relação a um processador não pipeline. Um processador com 7 estágios dentro do pipeline seria 7 vezes mais rápido que um não pipeline.
- Existe uma fórmula simples que mostra a vantagem da vazão (throughput) de instruções:

$$\text{pipeline time} = \frac{\text{no-pipeline time}}{\text{pipeline stage number}}$$

Performance do Pipeline

- Vamos considerar um exemplo em que tem-se 5000 instruções a serem executadas. Suponhamos um clock de 10 nano-segundos. Um processador não pipeline levaria $5000 * 10 = 50000$ ns.
- Se tivermos um processador com um pipeline de 12 estágios, temos que ao final da execução de todas instruções, o tempo decorrido seria $50000 / 12 = 4166.6$ ns.

Estágios do Pipeline (Modelo de 5 Estágios)

- Utilizaremos o modelo de 5 estágios: IF, ID, OF, IE, OS.

Etapa	Nome da Etapa	Descrição da Função
1. IF	Instruction Fetch (Busca)	Busca a instrução na Memória ou Cache.
2. ID	Instruction Decode (Decodificação)	Interpreta o Código de Operação (C.Op.).
3. OF	Operand Fetch (Busca de Operandos)	Localiza e busca os dados (operandos) nos Registradores.
4. IE	Instruction Execute (Execução)	A UAL ou FPU realiza o cálculo.
5. OS	Output Store (Armazenamento)	Armazena o resultado no Registrador ou Memória.

Execução Pipelined na Prática

- Ponto Chave: O tempo que cada estágio leva é, idealmente, um ciclo de clock.
- Paralelismo: No Ciclo 5, cinco instruções diferentes estão sendo processadas ao mesmo tempo, cada uma em um estágio distinto.

Tempo (Ciclo)	T1	T2	T3	T4	T5	T6
Instrução I1	IF	ID	OF	IE	OS	
Instrução I2		IF	ID	OF	IE	OS
Instrução I3			IF	ID	OF	IE

Desafios do Pipeline: Conflitos (Hazards)

- Conceito: Ocorre quando uma instrução no pipeline não pode ser executada no ciclo de clock planejado, causando uma pausa (stall) na linha de montagem.
- Tipos Principais:
 - Conflito de Dados (Data Hazard): Dependência de resultado.
 - Conflito de Controle (Control Hazard): Desvio de fluxo do programa.
 - Conflito Estrutural (Structural Hazard): Limitação de hardware.

Conflito de Dados (Data Hazard)

- Problema: Uma instrução (I2) precisa do resultado de uma instrução anterior (I1) que ainda não completou a fase de Armazenamento (OS).
- Exemplo:
 - ADD R1, R2, R3 (resultado em R1) seguido de SUB R4, R1, R5.
A SUB precisa de R1.
- Solução (Forwarding/Encaminhamento): A UCP detecta a dependência e encaminha o resultado diretamente da saída da UAL (IE) para a entrada da UAL da instrução seguinte, sem esperar o estágio OS.

Conflito de Controle (Control Hazard)

- Problema: Causado por instruções de desvio (JUMP, CALL, Branch). A UCP busca a próxima instrução (IF) antes de saber se o desvio será tomado.
- Custo: Se a UCP buscar o caminho errado, o pipeline inteiro precisa ser esvaziado (flush), causando vários ciclos perdidos.
- Solução (Predição de Desvio): O hardware tenta adivinhar se o desvio será tomado ou não, minimizando o custo do erro.

Conflito Estrutural (Structural Hazard)

- Problema: Dois estágios diferentes do pipeline tentam usar o mesmo recurso de hardware (ex: a mesma porta de leitura do banco de registradores ou o mesmo banco de memória) no mesmo ciclo de clock.
- Solução: Duplicação de recursos (ex: usar bancos de memória e cache separados para Instruções e Dados - Arquitetura Harvard).